PHASE 1 — 7-DAY WEEKLY SPRINT PLAN

> **Project:** AI-Powered Cloud Monitoring & Observability Platform
> **Team:** 3 developers — Rushikesh, Sayali, Pooja (using their existing track ownership from the long-term plan, applied at 7-day scale)
> **Stack (per project brief):** React + Vite + TypeScript (frontend) · Python FastAPI (backend) · Prometheus + Loki (monitoring) · LLM-based RCA (AI) · Docker-based local dev with a workload/traffic generator

1. Phase 1 Objective

In plain terms: by the end of Day 7, the team should be able to start a fake workload, watch it misbehave, and watch the platform notice, explain, and display that — for real, not as a mockup.

This sprint does **not** attempt the full platform. It exists to prove the riskiest, most load-bearing slice works end-to-end: **real metric → real detection → real incident → real (small) AI explanation → real dashboard**. Everything else waits.

2. Phase 1 Scope

IN SCOPE

- Local Docker environment: Postgres, Prometheus, a workload/traffic generator container that produces real CPU/memory load on demand.
- Basic JWT authentication (single user, login-protected dashboard) — just enough to gate the UI, not a full user-management system.
- A metrics proxy API that returns **real** Prometheus data (not mocked).
- A **threshold-based** anomaly check (e.g. "CPU > 80% for 30s") — not a trained ML model yet.
- Incident + Alert creation when a threshold breaches, persisted to Postgres.
- Incidents list + incident detail API and UI, showing the real evidence that triggered it.
- A **minimal** AI RCA: one real LLM call per incident, using the incident's own data as context, returning a small structured explanation. No knowledge base, no retrieval yet.
- A dashboard: login → live metric chart → incidents list → incident detail → AI explanation panel.
- Local-only deployment (docker compose up) — no cloud hosting yet.

OUT OF SCOPE (explicitly deferred)

- Trained ML anomaly detection (Isolation Forest / statistical models) — Phase 1 uses simple thresholds instead.
- RAG / vector store / embeddings / knowledge base — Phase 1's AI step has no retrieval step at all.
- Log aggregation (Loki) — metrics only in Phase 1.
- Multi-service/multi-container correlation — Phase 1 watches one workload target.
- Notifications (email/Slack/push).
- Historical incident analysis / reporting.
- Real user management, roles, password reset, etc. — single hardcoded/seeded login only.
- Cloud deployment (Vercel/Render) — local Docker only.

3. Architecture Foundation (minimum needed for Phase 1)

```
Workload Generator (Docker) ──produces load──► Prometheus (scrapes it)
                                                        │
                                          FastAPI queries Prometheus on demand
                                                        │
                                          ┌─────────────┴─────────────┐
                                          ▼                           ▼
                                  /metrics/query              Threshold checker
                                  (real data out)             (polls, on breach
                                                              creates Incident
                                                              + Alert in Postgres)
                                                                      │
                                                              /incidents API
                                                                      │
                                                              POST /incidents/:id/analyze
                                                                      │
                                                              LLM call (context-only,
                                                              no retrieval yet)
                                                                      │
                                                              React dashboard displays
                                                              metrics → incidents → RCA
```

No agent, no Loki, no Qdrant, no ML training pipeline in Phase 1 — those are real, but they're Phase 2+.

4. Developer Allocation

Developer 1 — Rushikesh — Telemetry & Platform Track

- **Module ownership:** Docker environment, workload generator, Prometheus setup, metrics proxy API, auth.
- **Backend:** docker-compose.yml (Postgres, Prometheus, workload container), POST/GET /auth, GET /api/v1/metrics/query.
- **Frontend:** Login screen, protected route wrapper.
- **Data/model:** none new (auth uses a single seeded user row).
- **Integration:** exposes the metrics endpoint Sayali's checker polls; exposes auth Pooja's frontend consumes.
- **Testing:** confirm Prometheus target is UP; confirm /metrics/query returns real, non-mocked values.
- **End-of-week result:** a running Docker stack where real load on the workload container is visibly scrapeable via a working, authenticated API.

Developer 2 — Sayali — Incident & Alerting Track

- **Module ownership:** threshold-based anomaly checking, incident/alert lifecycle, incidents API.
- **Backend:** Incident/Alert models + migration, the threshold-checker polling loop, GET /api/v1/incidents, GET /api/v1/incidents/:id.
- **Frontend:** Incidents list screen, Incident detail (evidence) screen.
- **Data/model:** incident schema includes the real metric value/threshold/timestamp that triggered it — this becomes the incident_context Pooja's AI step consumes.
- **Integration:** consumes Rushikesh's metrics endpoint; exposes incident_context shape to Pooja — agree this shape on Day 1, don't let it drift silently.
- **Testing:** manually spike the workload, confirm exactly one incident is created (no duplicate-spam), confirm evidence values are accurate.
- **End-of-week result:** triggering real load creates a real incident with real evidence, visible via API and UI.

Developer 3 — Pooja — AI RCA & Dashboard Shell Track

- **Module ownership:** minimal LLM-based RCA, overall frontend shell/routing, dashboard chart, AI RCA panel.
- **Backend:** prompt builder (plain function, no framework), LLM client wrapper, POST /api/v1/incidents/:id/analyze.
- **Frontend:** app shell/routing, Dashboard screen (live metric chart), AI RCA panel inside Incident Detail.

- **Data/model:** the RCA response JSON shape (summary, probable_cause, confidence, next_steps) — agree this with Sayali/Rushikesh on Day 1.
- **Integration:** consumes Sayali's incident_context; her analyze endpoint must degrade gracefully ("AI analysis unavailable") if the LLM call fails — never crash the incident view.
- **Testing:** confirm the analyze endpoint returns a real, schema-shaped response for a real incident; confirm graceful failure if the LLM key is missing/invalid.
- **End-of-week result:** a real incident, clicked in the UI, shows a real AI-generated explanation — not a hardcoded string.

Got it — here's the **Day-by-Day Plan** rewritten with that same beginner-friendly granularity: exact branch names, exact file paths, exact model fields, exact commands to run. This replaces Section 5 in the plan above; everything else stays the same.

5. Day-by-Day Plan (Detailed)

DAY 1: Auth Foundation, Incident Schema & Frontend Shell

Rushikesh — Task: Docker Infra, Auth Model & Login API (Module: Platform Foundation)

- **Branch**: feature/rushikesh-day1-infra-auth
- **Database Model (backend/app/models/user.py)**: Define User model — id, email, password_hash, role (default "engineer"), created_at.
- **Backend Files**:
  - backend/app/core/config.py — Settings class loading .env (DB URL, JWT secret).
  - backend/app/core/database.py — SQLAlchemy engine + session setup.
  - backend/app/core/security.py — password hashing (bcrypt) + JWT create/verify helpers.
  - backend/app/api/v1/auth.py — stub only today (POST /api/v1/auth/login — logic tomorrow).
- **Infra Files**:
  - docker-compose.yml — services: db (Postgres), prometheus, workload-generator.
  - infrastructure/prometheus/prometheus.yml — scrape config targeting workload-generator.
  - infrastructure/workload-generator/Dockerfile + app.py — a tiny Flask/FastAPI app exposing POST /spike that burns CPU for N seconds so we have something real to monitor.
- **Command to run**: docker compose up --build -d then alembic revision --autogenerate -m "init users table" → alembic upgrade head.

Sayali — Task: Incident & Alert Schema + Threshold Logic (Module: Detection Foundation)

- **Branch**: feature/sayali-day1-incident-schema
- **Database Model (backend/app/models/incident.py)**: Define Incident model — id, metric_name, metric_value (Float), threshold (Float), severity ("warning"/"critical"), status (default "open"), triggered_at, resolved_at (nullable). Define Alert model — id, incident_id (FK), message, created_at.
- **Backend Files**:
  - backend/app/api/v1/incidents.py — stub router, GET /api/v1/incidents returns [] for now.
  - backend/app/services/detection/threshold_checker.py — **pure function**: def is_breach(value: float, threshold: float) -> bool. No live data wiring yet — this is deliberately isolated so it's testable today without Rushikesh's API existing.
- **Test File**: backend/tests/test_threshold_checker.py — assert is_breach(85, 80) == True, is_breach(50, 80) == False.
- **Command to run**: alembic revision --autogenerate -m "add incidents and alerts tables" → alembic upgrade head, then pytest backend/tests/test_threshold_checker.py.

Pooja — Task: Frontend Scaffold & AI Skeleton (Module: Dashboard Shell)

- **Branch**: feature/pooja-day1-frontend-shell
- **Frontend Setup**: npm create vite@latest frontend -- --template react-ts, install axios, react-router-dom, recharts.
- **Frontend Files**:
  - frontend/src/routes/AppRoutes.tsx — route table: /login, /dashboard, /incidents, /incidents/:id.
  - frontend/src/layouts/DashboardLayout.tsx — sidebar + header shell (no real data yet).
  - frontend/src/pages/LoginPage.tsx — placeholder form (wired to real API tomorrow).
  - frontend/src/styles/tokens.css — color palette + spacing variables so all screens look consistent from day one.
- **Backend Files** (skeleton only, no logic):
  - backend/app/services/ai/prompt_builder.py — empty def build_prompt(...): pass.
  - backend/app/services/ai/llm_client.py — empty def call_llm(prompt: str): pass.
- **Command to run**: cd frontend && npm install && npm run dev — confirm it loads at localhost:5173.

DAY 2: Real Auth, Metrics Proxy & Login Wiring

Rushikesh — Task: Real Login API & Metrics Proxy (Module: Auth + Ingestion)

- **Branch**: feature/rushikesh-day2-auth-metrics
- **Backend Files**:
  - backend/app/api/v1/auth.py — implement POST /api/v1/auth/login (verify password, return JWT).
  - backend/app/services/ingestion/prometheus_client.py — def query_range(metric: str, minutes: int) -> list[dict] calling Prometheus's HTTP API.
  - backend/app/api/v1/metrics.py — GET /api/v1/metrics/query?metric=cpu&range=5m, returns real data via prometheus_client.py.
- **Command to run**: uvicorn app.main:app --reload, then curl http://localhost:8000/api/v1/metrics/query?metric=cpu&range=5m — confirm real numbers come back, not a placeholder.

Sayali — Task: Scheduler Skeleton (Module: Detection Foundation)

- **Branch**: feature/sayali-day2-scheduler-skeleton
- **Backend Files**:
  - backend/app/services/detection/scheduler.py — set up APScheduler, register a job that runs every 15s (job body is a pass stub today — wired to real data tomorrow).
- **Test File**: backend/tests/test_scheduler.py — confirm the scheduler starts and the job fires at least once within 20s.
- **Command to run**: pytest backend/tests/test_scheduler.py.

Pooja — Task: Login Screen Wired to Real API (Module: Auth UI)

- **Branch**: feature/pooja-day2-login-wiring
- **Frontend Files**:
  - frontend/src/pages/LoginPage.tsx — real form calling POST /api/v1/auth/login.
  - frontend/src/context/AuthContext.tsx — stores JWT in memory, exposes login()/logout().
  - frontend/src/components/ProtectedRoute.tsx — redirects to /login if no token.
- **Command to run**: npm run dev — log in with the seeded user, confirm redirect to /dashboard.

DAY 3: Metrics on the Dashboard + Detection Goes Live

Rushikesh — Task: Metrics Range Support & Health Check (Module: Ingestion Polish)

- **Branch**: feature/rushikesh-day3-metrics-range-health

- **Backend Files**:
  - backend/app/api/v1/metrics.py — add configurable time-range query param (5m, 15m, 1h).
  - backend/app/api/v1/health.py — GET /api/v1/system/health checks DB + Prometheus reachability, returns {"db": "ok", "prometheus": "ok"}.
- **Command to run**: curl http://localhost:8000/api/v1/system/health.

Sayali — Task: Wire Detection to Real Metrics (Module: Live Threshold Detection)

- **Branch**: feature/sayali-day3-live-detection
- **Backend Files**:
  - backend/app/services/detection/threshold_checker.py — now calls Rushikesh's metrics.py service, uses is_breach() from Day 1, and on breach creates a real Incident + Alert row.
  - backend/app/repositories/incident_repository.py — create_incident(...), get_open_incident_for_metric(...) (prevents duplicate spam).
- **Command to run**: hit POST /spike on the workload-generator, watch a real row appear: psql or SELECT * FROM incidents;.

Pooja — Task: Live Dashboard Chart (Module: Dashboard)

- **Branch**: feature/pooja-day3-dashboard-chart
- **Frontend Files**:
  - frontend/src/pages/DashboardPage.tsx — page layout.
  - frontend/src/components/MetricChart.tsx — Recharts line chart, polls /api/v1/metrics/query every 5s.
  - frontend/src/api/metricsApi.ts — getMetrics(metric, range) fetch wrapper.
- **Command to run**: npm run dev — trigger a spike, watch the chart move live.

DAY 4: Incidents API & Incidents UI

Rushikesh — Task: Stabilize + Documentation (Module: Support)

- **Branch**: feature/rushikesh-day4-stabilize-docs
- **Files**: README.md — write the exact local setup steps (clone → .env → docker compose up --build → alembic upgrade head → npm run dev).
- **Command to run**: docker compose down -v && docker compose up --build — confirm a truly clean start works, following your own README.

Sayali — Task: Incidents List & Detail API (Module: Incidents API)

- **Branch**: feature/sayali-day4-incidents-api
- **Backend Files**:
  - backend/app/schemas/incident.py — Pydantic IncidentOut schema (id, metric_name, metric_value, threshold, severity, status, triggered_at).
  - backend/app/api/v1/incidents.py — implement GET /api/v1/incidents (list, optional ?status=open) and GET /api/v1/incidents/{id}.
- **Command to run**: pytest backend/tests/test_incidents_api.py.

Pooja — Task: Incidents List & Detail Screens (Module: Incidents UI)

- **Branch**: feature/pooja-day4-incidents-ui
- **Frontend Files**:
  - frontend/src/pages/IncidentsListPage.tsx — table of incidents (severity badge, timestamp).
  - frontend/src/pages/IncidentDetailPage.tsx — shows the real evidence (metric, value, threshold).

- frontend/src/api/incidentsApi.ts — getIncidents(), getIncidentById(id).

- **Command to run**: npm run dev — click into a real incident, confirm evidence matches the DB row.

DAY 5: Minimal AI Root Cause Analysis

Rushikesh — Task: LLM Call Infrastructure (Module: AI Infra Support)

- **Branch**: feature/rushikesh-day5-llm-infra
- **Backend Files**:
  - backend/app/core/config.py — add LLM_API_KEY, LLM_TIMEOUT_SECONDS.
  - backend/app/services/ai/llm_client.py — implement call_llm(prompt: str) -> dict with a timeout + one retry, raising a clear LLMUnavailableError on failure (never a silent crash).
- **Command to run**: pytest backend/tests/test_llm_client.py (mock the API call).

Sayali — Task: Build the incident_context Contract (Module: Correlation Foundation)

- **Branch**: feature/sayali-day5-incident-context
- **Backend Files**:
  - backend/app/services/correlation/incident_context.py — def build_incident_context(incident) -> dict, returning {"metric_name", "metric_value", "threshold", "triggered_at", "service": "workload-generator"}.
- **Test File**: backend/tests/test_incident_context.py.
- **Command to run**: pytest backend/tests/test_incident_context.py.

Pooja — Task: Prompt Builder & Analyze Endpoint (Module: AI RCA)

- **Branch**: feature/pooja-day5-ai-analyze-endpoint
- **Backend Files**:
  - backend/app/services/ai/prompt_builder.py — build_prompt(incident_context: dict) -> str.
  - backend/app/schemas/rca.py — Pydantic RCAOut (summary, probable_cause, confidence, next_steps).
  - backend/app/api/v1/ai.py — POST /api/v1/incidents/{id}/analyze — builds context → builds prompt → calls LLM → validates against RCAOut → returns it.
- **Command to run**: curl -X POST http://localhost:8000/api/v1/incidents/1/analyze — confirm a real, structured JSON response.

DAY 6: Wire AI RCA into the UI + System Status

Rushikesh — Task: README Finalization & System Health Screen Support (Module: Support)

- **Branch**: feature/rushikesh-day6-readme-health
- **Files**: README.md — add troubleshooting section; backend/app/api/v1/health.py — expand response to include Prometheus target status.
- **Command to run**: fresh clone in a new folder, follow the README exactly, confirm zero manual fixes needed.

Sayali — Task: Incident Status Transitions (Module: Incidents Polish)

- **Branch**: feature/sayali-day6-incident-status
- **Backend Files**: backend/app/api/v1/incidents.py — add PATCH /api/v1/incidents/{id}/status (open → resolved).
- **Command to run**: pytest backend/tests/test_incidents_api.py.

Pooja — Task: AI RCA Panel UI (Module: AI RCA UI)

- **Branch**: feature/pooja-day6-ai-rca-panel
- **Frontend Files**:

- frontend/src/components/AiRcaPanel.tsx — "Analyze" button, loading spinner, renders summary/probable_cause/confidence/next_steps.
- frontend/src/api/aiApi.ts — analyzeIncident(id).
- Handle the failure case explicitly: if the call fails, show **"AI analysis unavailable"** — never a blank screen or crash.

- **Command to run**: npm run dev — click Analyze on a real incident, confirm the panel renders; then temporarily break the LLM key and confirm the graceful fallback shows instead of crashing.

DAY 7: Bug Bash & Demo Rehearsal

> **All Three — Task: Full Flow Verification**

- **Branch naming for any fixes**: feature/<name>-day7-bugfix-<short-description>
- **Activity**: Run the full demo (Section 8) at least 3 times, in front of each other. Fix anything flaky (duplicate incidents, chart not updating, AI panel hanging).
- **Command to run (final check)**: docker compose down -v && docker compose up --build → npm run dev → run the full spike → dashboard → incident → analyze flow end-to-end, twice, timed.

6. Parallel Dependency Analysis

Can start immediately, no dependencies:

- Rushikesh: Docker Compose skeleton, workload generator, Prometheus config.
- Sayali: DB models/migration, pure threshold-check logic (unit-testable in isolation).
- Pooja: Frontend scaffold, routing, design tokens, prompt-builder skeleton (no live data needed to start the shape).

Real dependencies (the only ones that matter):

- Sayali's threshold checker needs Rushikesh's real metrics endpoint → ready by end of Day 2.
- Pooja's dashboard chart needs the same → ready by end of Day 2.
- Pooja's AI RCA needs Sayali's incident_context shape → **agree the shape on Day 1**, so Pooja can build the prompt builder against a fixed contract even before real incidents exist (use a fixture/sample incident to develop against).
- Everything else is additive polish, not a hard blocker.

Contracts that must be agreed before Day 1 ends:

1. Metrics response shape (Rushikesh defines, others consume).
2. incident_context shape (Sayali defines, Pooja consumes).
3. RCA response JSON shape (Pooja defines, Sayali/Rushikesh's UI/DB consume).

How to prevent blocking: each person can build against the *agreed shape* using fixture/fake data before the real upstream piece exists — nobody waits idle for someone else to finish.

7. Git / Branch Ownership

main — not touched during Phase 1

└ dev — integration branch, all PRs land here

├ feature/rushikesh-day<N>-<module> e.g. feature/rushikesh-day1-docker-infra

├ feature/sayali-day<N>-<module> e.g. feature/sayali-day3-threshold-checker

└ feature/pooja-day<N>-<module> e.g. feature/pooja-day5-ai-rca

Shared files to modify carefully (flag in PR before touching):

- docker-compose.yml — Rushikesh owns; others propose changes via PR comment first.
- backend/app/main.py (router registration) — whoever adds a new router touches one line here; keep these commits small and separate from logic changes to avoid conflicts.

- The three agreed JSON contracts (Section 6) — any shape change gets flagged to the consuming owner *before* merge, not discovered after.

8. Phase 1 End-to-End Demo (Day 7)

```
Team starts the workload generator and tells it to spike CPU
↓
Prometheus scrapes the real spike within seconds
↓
Backend's threshold checker polls, sees the breach
↓
A real Incident + Alert row is created in Postgres
↓
Dashboard chart visibly shows the spike (real data, live)
↓
Incidents List shows the new incident appear
↓
Team clicks into Incident Detail — sees the real evidence
↓
Team clicks "Analyze" — a real LLM call runs
↓
AI RCA panel shows a real, structured explanation
↓
Team explains: "this is the seed of the full platform;
here's what's real vs. what's coming in Phase 2"
```

Every step above uses real data — no step in this demo is faked or hardcoded.

9. Definition of Done

- [ ] docker compose up --build succeeds from a clean clone, on any of the 3 developers' machines.
- [ ] The workload generator can be told to spike CPU/memory on command.
- [ ] Prometheus shows the workload target as UP and scrapes real values.
- [ ] GET /api/v1/metrics/query returns real, current Prometheus data (verified by comparing to Prometheus's own UI).
- [ ] Login works with the seeded user; unauthenticated requests are rejected.
- [ ] Spiking the workload reliably creates exactly one real Incident row (no duplicates, no misses) within the polling interval.
- [ ] GET /api/v1/incidents and GET /api/v1/incidents/:id return real data matching what's in Postgres.
- [ ] The dashboard chart updates with real data when the workload spikes — no mock numbers anywhere in the frontend.
- [ ] Clicking "Analyze" on a real incident returns a real, schema-shaped LLM response — not a hardcoded string.
- [ ] If the LLM call fails (bad/missing key), the UI shows "AI analysis unavailable" instead of crashing.
- [ ] All three developers can independently reproduce the full demo from a clean docker compose up.
- [ ] No critical integration blockers remain between any two developers' work.

10. Future Phases (list only — not planned in detail)

Phase 2: Replace threshold detection with a real trained anomaly model (Isolation Forest); add log collection (Loki) and correlate logs+metrics into incidents; add a real RAG knowledge base (Qdrant + embeddings) feeding the AI RCA step.

Phase 3: Multi-service/multi-container visibility; incident lifecycle states (investigating/resolved); notifications; historical incident analysis.

Phase 4: Real authentication/user management (roles, multiple users); cloud deployment (Vercel + Render); production hardening (CORS, secrets, rate limiting); CI/CD pipeline.